

INTERNSHIP REPORT / RAPPORT DE STAGE

Log Anomaly Detection with Hawkes-Process Timing and LLM Triage on System Logs

An unsupervised, rigorously validated pipeline for security-log analysis,
developed during an internship at OCP Group

Author: El Alami Amine
Host organisation: OCP Group — [host entity / department, e.g. Digital
& Information Systems, Cybersecurity]
Company supervisor: [Company supervisor's name]
Academic supervisor: [Academic supervisor's name]
Internship period: [Start date] – [End date], 2026

Acknowledgements

I would like to express my sincere gratitude to my company supervisor, [Company supervisor's name], for the guidance, trust, and autonomy granted throughout this internship, and to the members of the OCP Group — [host entity / department, e.g. Digital & Information Systems, Cybersecurity] team for their welcome and for the many discussions that helped anchor an academic project in operational reality. I also thank my academic supervisor, [Academic supervisor's name], for the methodological guidance and for encouraging the emphasis placed on rigorous validation. Finally, I thank OCP Group for providing an environment in which an intern is encouraged to pursue technical depth, and my professors at [Your school / university name] for the mathematical foundations on which this work rests.

Abstract

This report documents a two-month internship project carried out within OCP Group: the design, implementation, and rigorous validation of an end-to-end anomaly-detection system for machine-generated system logs, augmented with a stochastic-process model of event timing and a large-language-model (LLM) triage layer. The system is developed and evaluated entirely on public benchmark datasets (`loghub` HDFS and BGL), which makes every result in this report reproducible while remaining directly relevant to the security-operations context of a large industrial group. The pipeline comprises: (i) automatic log parsing into structured events (Drain); (ii) an unsupervised autoencoder detector benchmarked against an Isolation Forest baseline; (iii) a univariate Hawkes point process fitted by maximum likelihood to model the self-excitation of log events, validated with the time-rescaling theorem, and evaluated as a source of detection features; (iv) an adaptive alerting threshold based on extreme-value theory and a distribution-free alternative, designed to stabilise the false-alarm rate under distribution drift; and (v) an LLM triage agent that deduplicates alerts, correlates them into incidents, and produces ranked, structured, explained incident reports, surfaced in a Streamlit dashboard. Validation follows a deliberately strict protocol: a never-shuffled temporal train/test split, precision–recall analysis rather than accuracy, a headline metric of false alarms per day at a fixed detection rate, and explicit measurement of performance drift. Key results include a detection PR-AUC of 0.987 on HDFS (a $\sim 20\times$ reduction in false alarms relative to the baseline), a near-critical Hawkes branching ratio ($\eta \approx 0.97\text{--}1.0$) on BGL, a measured halving of false alarms attributable to the timing features, and a demonstrated stabilisation of the false-alarm rate under drift. Negative and nuanced findings are reported with the same care as positive ones, as an explicit methodological choice.

Keywords: anomaly detection, system logs, Hawkes process, point processes, extreme-value theory, unsupervised learning, autoencoder, LLM, security operations, OCP.

Contents

Acknowledgements	1
Abstract	2
Acronyms	7
1 General introduction	8
1.1 Context of the internship	8
1.2 Why this problem matters to OCP	8
1.3 Objectives	9
1.4 Structure of the report	9
2 Presentation of the host organisation: OCP Group	10
2.1 Overview and history	10
2.2 Industrial footprint	10
2.3 Strategy, research, and the skills ecosystem	11
2.4 Digital infrastructure and operations	11
2.5 Positioning of the internship	11
3 Problem statement and state of the art	13
3.1 Formal problem statement	13
3.2 State of the art	13
3.3 Positioning: prior art versus contribution	14
4 Theoretical background	15
4.1 Evaluation under extreme class imbalance	15
4.2 Temporal validation and leakage	15
4.3 Unsupervised detectors	16
4.3.1 Isolation Forest	16
4.3.2 Autoencoders	16
4.4 Point processes and the Hawkes model	16
4.4.1 Counting processes and conditional intensity	16
4.4.2 The Hawkes process	16
4.4.3 Likelihood and estimation	17
4.4.4 Goodness of fit: the time-rescaling theorem	17
4.4.5 Simulation: Ogata thinning	17
4.5 Extreme-value theory for thresholds	18
5 Methodology and system design	19
5.1 Datasets	19
5.2 Pipeline architecture	19
5.3 Log parsing	19

5.4	Feature construction	20
5.5	Validation protocol	20
5.6	Engineering and quality practices	20
6	Implementation and results	21
6.1	Data acquisition and parsing	21
6.2	Exploratory analysis: measurements that shaped the design	21
6.3	Detection on HDFS	22
6.4	The Hawkes timing model on BGL	23
6.5	Ablation: do timing features improve detection?	24
6.6	Adaptive thresholding under drift	25
6.7	Triage: from alerts to ranked incidents	26
6.8	Demonstrator dashboard	27
7	Discussion	29
7.1	Synthesis of results	29
7.2	Limitations	29
7.3	Skills developed during the internship	30
8	Conclusion and perspectives	31
8.1	Conclusion	31
8.2	Perspectives	31
A	Notation	33
B	Reproducibility	34

List of Figures

6.1	Left: HDFFS event volume per minute—a flat baseline punctuated by bursts, the visual signature of self-excitation. Right: block-session sizes (median 19, maximum 298).	22
6.2	Precision–recall curves on the HDFFS test split. The autoencoder dominates the baseline at every operating point; the dashed line marks the chance level (0.022).	23
6.3	Top: fitted conditional intensity $\lambda(t)$ on the calm window —near-zero baseline with self-exciting spikes. Bottom: time-rescaling QQ-plots against $\text{Exp}(1)$ for the calm (left) and cascade (right) windows: the body lies on the diagonal, the upper tail departs.	24
6.4	Precision–recall curves for the ablation. The content-only curve collapses to the chance line at moderate recall; the timing-based curves retain $\approx 2\times$ chance precision across the operating range.	25
6.5	Top: fixed versus adaptive threshold against the drifting score distribution. Bottom: resulting false alarms per day; the fixed threshold explodes mid-period while the adaptive threshold holds the alert budget.	26
6.6	The Incidents view of the demonstrator: 200 raw flags reduced to 90 ranked incidents with priorities, categories, and recommended actions.	28

List of Tables

3.1	Prior art versus this project’s contribution, by component.	14
5.1	Datasets and their roles.	19
6.1	HDFS detection, temporal test split (172,518 blocks, 2.16% anomalous).	22
6.2	Maximum-likelihood Hawkes fits on BGL windows.	23
6.3	Feature ablation on BGL windows (test split, 4.71% anomalous).	25
6.4	False-alarm stability under drift (HDFS autoencoder scores, $q = 3\%$).	26
7.1	Consolidated results of the internship project.	29

Acronyms

AE	Autoencoder
API	Application Programming Interface
BGL	Blue Gene/L (supercomputer log dataset)
EDA	Exploratory Data Analysis
EVT	Extreme-Value Theory
FA	False Alarm
GPD	Generalized Pareto Distribution
HDFS	Hadoop Distributed File System (log dataset)
IT / OT	Information Technology / Operational Technology
KS	Kolmogorov–Smirnov (test)
LLM	Large Language Model
MLE	Maximum-Likelihood Estimation
MLP	Multilayer Perceptron
OCP	OCP Group (historically, Office Chérifien des Phosphates)
POT	Peaks-Over-Threshold
PR-AUC	Area Under the Precision–Recall Curve
SOC	Security Operations Centre
UM6P	Mohammed VI Polytechnic University

Chapter 1

General introduction

1.1 Context of the internship

This report presents the work carried out during a two-month internship within OCP Group — [host entity / department, e.g. Digital & Information Systems, Cybersecurity]. The internship sits at the intersection of two subjects of direct relevance to a large industrial group: the *operational security* of digital infrastructure, and the *statistical modelling of event data*. Modern industrial organisations operate thousands of servers, applications, and industrial control systems, all of which continuously emit *logs*: timestamped text records of their activity. These logs are simultaneously the primary forensic record of the infrastructure and an overwhelming torrent of data—millions of lines per day—in which the events that matter (failures, misconfigurations, attacks, cascading faults) are rare, unlabelled, and buried.

The project addresses this problem end to end: it builds a system that ingests raw logs and produces a short, ranked, explained list of incidents. Beyond the engineering, the internship had an explicit academic objective, agreed with both supervisors: to treat the problem as a case study in *rigorous time-series methodology*—unsupervised learning under class imbalance, point-process modelling of event timing, extreme-value methods for alerting, and a validation discipline strict enough to be defensible in front of a critical audience.

1.2 Why this problem matters to OCP

OCP Group’s industrial and commercial operations rest on a large digital estate (Chapter 2). For any organisation of this scale, the security-operations workload has a well-known structure: monitoring systems generate far more alerts than analysts can individually review, most alerts are false or redundant, and genuine incidents manifest as *bursts* of correlated events rather than isolated signals. The three components of this project map directly onto that reality: unsupervised detection addresses the absence of labelled attacks; the Hawkes timing model addresses the bursty, cascading nature of real incidents; and the triage layer addresses alert fatigue by converting raw detections into a small number of prioritised, explained incidents.

Scope and data

For confidentiality and reproducibility, the entire project was developed and evaluated on *public* benchmark datasets (the `loghub` collection: HDFS and BGL), not on OCP internal data. This was a deliberate design choice made at the outset: it allows every figure and table in this report to be reproduced from the accompanying repository, while the methods themselves are directly transferable to any log source. No internal OCP system, dataset, or credential was used or is described in this report.

1.3 Objectives

The objectives fixed at the start of the internship were:

1. Build a complete, reproducible pipeline: raw logs $\rightarrow$ structured events $\rightarrow$ unsupervised detection $\rightarrow$ calibrated alerting $\rightarrow$ triage $\rightarrow$ dashboard.
2. Place a *stochastic-process model* at the mathematical core: fit a Hawkes process to event timestamps, estimate its branching ratio, validate the fit with the time-rescaling theorem, and evaluate the resulting timing features for detection.
3. Apply a strict validation protocol throughout: temporal train/test split, precision–recall analysis, false alarms per day as the headline metric, and drift measurement.
4. Prototype an LLM-based triage agent producing structured, ranked, explained incidents, and a dashboard demonstrator.
5. (Stretch) Investigate extreme-value theory for self-calibrating alert thresholds.

All five objectives, including the stretch goal, were completed; the results are presented in Chapters 6–7.

1.4 Structure of the report

Chapter 2 presents the host organisation, its digital context, and the positioning of the internship. Chapter 3 defines the problem precisely and reviews the state of the art, separating established prior work from this project’s contributions. Chapter 4 provides the theoretical background—each mathematical tool is introduced from first principles. Chapter 5 describes the methodology: datasets, system architecture, validation protocol, and engineering practices. Chapter 6 presents the implementation and results of every stage, from parsing to the dashboard, including the measurements that redirected the design. Chapter 7 discusses limitations and the skills acquired. Chapter 8 concludes and outlines perspectives.

Chapter 2

Presentation of the host organisation: OCP Group

2.1 Overview and history

OCP Group (historically *Office Chérifien des Phosphates*) is a Moroccan public limited company and one of the world’s leading producers and exporters of phosphate rock, phosphoric acid, and phosphate-based fertilizers. Founded in 1920 to mine the phosphate deposits of the Moroccan plateau, the organisation has evolved over a century from a mining office into a vertically integrated industrial group covering the entire phosphate value chain—extraction, beneficiation, chemical processing, and global distribution. In 2008 it became a société anonyme (OCP S.A.), a transformation that accompanied a broad programme of industrial modernisation and internationalisation.

Phosphate is a strategic resource: it is the primary source of phosphorus, one of the three essential macronutrients in agriculture, and therefore a cornerstone of global food security. Morocco holds the large majority of the world’s known phosphate rock reserves, and OCP, as their operator, plays a correspondingly central role both in the national economy—as one of Morocco’s largest companies and export earners—and in the worldwide fertilizer supply chain, serving customers across the Americas, Europe, Asia, and, with particular strategic emphasis, Africa.

2.2 Industrial footprint

OCP’s operations are organised around integrated mining and processing axes:

- **Mining sites.** The principal mining basins are *Khouribga* (the group’s largest), the *Gantour* basin (Benguerir and Youssoufia), and *Boucraa*. Ore is extracted, beneficiated (washing, flotation, drying), and routed to the chemical platforms.
- **Chemical and port platforms.** The *Jorf Lasfar* complex on the Atlantic coast—among the largest fertilizer production sites in the world—and the *Safi* complex transform phosphate rock into phosphoric acid and finished fertilizers for export.
- **The slurry pipeline.** A flagship of the group’s modernisation, a pipeline of roughly 187 km transports phosphate slurry from Khouribga to Jorf Lasfar, replacing rail transport, reducing cost and water/energy intensity per tonne, and illustrating the group’s strategy of infrastructure-led efficiency.

Around this industrial core, the group has built an ecosystem of subsidiaries and partnerships covering engineering, digital technology, research, and education, discussed below.

2.3 Strategy, research, and the skills ecosystem

Two strategic axes of the group are directly relevant to this internship’s context. The first is **sustainability**: flagship programmes target the massive use of desalinated and treated water, large-scale renewable energy for industrial operations, and “green” fertilizer value chains including green ammonia. The second is **knowledge and talent**: in 2017 the group founded *Mohammed VI Polytechnic University* (UM6P) in Benguerir, a research university oriented toward applied science, engineering, and industrial innovation, closely linked to the group’s R&D agenda. Through UM6P and associated programmes, OCP has positioned itself not only as an industrial operator but as an anchor of Morocco’s scientific and digital skills ecosystem—the environment in which internships such as this one take place.

2.4 Digital infrastructure and operations

OCP has pursued, over the last decade, a broad **digital transformation** of its industrial and corporate operations. Without entering into internal specifics—which are outside the scope and clearance of this report—the publicly visible pillars of this transformation, and their consequences for security operations, can be summarised as follows.

From mine to port: an instrumented value chain. Modern phosphate production is heavily instrumented: sensors and industrial control systems monitor extraction, beneficiation, the slurry pipeline, and chemical processing. This *operational technology* (OT) estate coexists with a large corporate *information technology* (IT) estate—enterprise resource planning, logistics, commerce, engineering, and collaboration systems. The group has invested in dedicated digital structures and partnerships to industrialise this transformation, including joint ventures and entities focused on IT services and digital innovation (for example, a joint venture with IBM—Teal Technology Services—created to support the group’s IT modernisation, and digital innovation structures associated with UM6P).

Why this creates a log-analysis problem. Every layer of such an estate—servers, network equipment, applications, industrial gateways, security appliances—continuously emits logs. At industrial scale this means, conservatively, millions of log lines per day, in heterogeneous formats, with no labels indicating which lines matter. Security and reliability teams face three structural difficulties, none specific to OCP but all amplified by scale:

1. **Volume asymmetry**: the events worth human attention are a vanishing fraction of the stream (10^{-3} or less), so manual review is impossible and naive alerting drowns analysts in false positives.
2. **Absence of labels**: new failure modes and attack techniques appear continuously; no labelled training catalogue can be exhaustive, which pushes the state of the art toward *unsupervised* methods that model normality.
3. **Temporal clustering**: real incidents are *cascades*—one root cause triggers bursts of correlated events across systems. Timing structure is therefore informative, yet most deployed tooling treats events as independent.

These three difficulties are precisely the three axes of this project (unsupervised detection; timing modelling; triage/correlation), which is what makes an academically oriented project directly relevant to the operational context of the host organisation.

2.5 Positioning of the internship

The internship was hosted in OCP Group — [host entity / department, e.g. Digital & Information Systems, Cybersecurity]. The role of the intern was to design, implement, validate, and document

the complete prototype described in this report, working autonomously with regular supervision checkpoints. As stated above, the prototype was developed on public benchmark data so that the work is fully reproducible and publishable internally without touching production systems; the methods, however, were selected explicitly for their transferability to an industrial SOC context: every component consumes generic structured logs and produces artefacts (scores, calibrated alerts, ranked incidents) that map onto the daily workflow of a security-operations team.

Chapter 3

Problem statement and state of the art

3.1 Formal problem statement

Let a log source emit a stream of raw text lines. After parsing (Section 5.3), the stream becomes a sequence of events $\{(t_i, e_i)\}_{i \geq 1}$, where t_i is a timestamp and $e_i \in \{1, \dots, K\}$ is a categorical *event type* drawn from a finite vocabulary of K templates. Events are grouped into *units of analysis* u (sessions or fixed time windows), each with an unknown binary state $y_u \in \{0, 1\}$ (normal/anomalous). The task is: using *no* labels at training time, learn a scoring function $s(u) \in \mathbb{R}$ such that anomalous units receive systematically higher scores, then convert scores to alerts via a threshold whose false-alarm behaviour is controlled, and finally aggregate alerts into a small set of ranked, explained incidents. Ground-truth labels are used exclusively for evaluation.

Three properties of the domain constrain any solution: extreme class imbalance ($\Pr[y_u = 1] \sim 10^{-3}$ – 10^{-1} depending on the unit); non-stationarity (the distribution of normal activity drifts over hours and days); and temporal clustering of events (incidents are cascades). The evaluation protocol (Section 5.5) is designed around exactly these three properties.

3.2 State of the art

Log parsing. The conversion of free-text lines into a finite template vocabulary is a mature problem. **Drain** [1] is a fixed-depth parse-tree algorithm that assigns each incoming line to a template cluster in a single online pass; it is the de-facto standard, distributed with per-dataset benchmark configurations in the **logparser** toolkit and evaluated across sixteen datasets in the **loghub** benchmark [2].

Log anomaly detection. Classical approaches construct a per-session *event-count matrix* and apply unsupervised detectors—PCA, invariant mining, clustering, or Isolation Forest [6]—as systematised in the **loglizer** benchmark [3]. Deep approaches model event *sequences*: DeepLog [4] treats logs as a language and trains an LSTM to predict the next event, flagging low-probability continuations; LogAnomaly [5] extends this with template semantics. Autoencoders on count vectors are a standard unsupervised baseline. On the HDFS benchmark, published methods routinely attain F1 or PR-AUC in the 0.95–0.99 range; the result is heavily reproduced.

Point processes. Hawkes processes [7] are the canonical model of self-exciting event streams, with mature estimation theory (maximum likelihood with the exponential-kernel recursion), simulation (Ogata thinning [8]), and goodness-of-fit machinery (the time-rescaling theorem [9]). They are standard in seismology, financial microstructure, and social-media modelling. Their use as a *feature source* for log anomaly detection is, by contrast, uncommon: the dominant paradigm discards fine-grained timing entirely.

Adaptive thresholding. Extreme-value theory—the Pickands–Balkema–de Haan theorem and the Peaks-Over-Threshold method [10]—provides the principled route from anomaly scores to thresholds with controlled exceedance probability. Its streaming application to anomaly detection is the SPOT/DSPOT family [11].

LLM-based triage. The use of large language models to summarise, classify, and prioritise security alerts is an active but young area, largely driven by industrial practice; the emerging consensus is that LLMs are most reliable when constrained to structured output schemas and applied *after* deterministic reduction (deduplication, correlation) of the alert volume.

3.3 Positioning: prior art versus contribution

Table 3.1 states explicitly, for each component, what was already established and what this project adds. Two points deserve emphasis. First, the HDFS detection result obtained in this work (PR-AUC 0.987) is a *reproduction* of a well-known benchmark outcome; it is claimed here only as evidence that the pipeline is correctly built, and as the substrate on which the novel components operate. Second, the project’s centre of gravity is elsewhere: the timing model and its honest evaluation, the validation protocol, the drift-aware thresholding, and the correlation-grounded triage.

Table 3.1: Prior art versus this project’s contribution, by component.

Component	Established prior work	Contribution of this project
Parsing	Drain, standard configurations [1].	Used unchanged, for comparability.
Detection	Count-vector AE / IsoForest on HDFS widely reproduced (0.95–0.99) [3, 4].	Reproduced as a validated baseline; the added value is the strict protocol and drift analysis around it.
Timing	Hawkes theory and diagnostics mature in other fields [7, 9].	Applied to logs: branching-ratio estimation on BGL, time-rescaling validation, and a measured ablation of timing features for detection.
Thresholding	EVT/POT theory [10]; SPOT for streams [11].	Implemented and verified; documented failure mode on discrete scores; drift-stabilisation demonstration.
Triage	LLM alert summarisation (emerging practice).	Correlation grounded in self-excitation; strict JSON schema; dedup-before-LLM design.
Validation	Temporal splits and PR analysis known in principle.	Applied end-to-end without exception; false-alarms/day headline; per-period drift reporting.

Chapter 4

Theoretical background

This chapter presents, from first principles, the mathematical tools used in the project. The aim is that a reader with a general scientific background can follow every later chapter without external references.

4.1 Evaluation under extreme class imbalance

Let a scored test set contain P anomalous and N normal units with $P \ll N$. For any threshold on the score, the confusion counts (TP, FP, FN, TN) define

$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}.$$

Accuracy = $(\text{TP} + \text{TN})/(P + N)$ is dominated by TN when $P \ll N$: a degenerate detector that never alerts attains accuracy $N/(N + P) \approx 1$. It is therefore not reported in this work. Sweeping the threshold traces the precision–recall curve; its area (**PR-AUC**, equivalently average precision) summarises ranking quality. Its baseline value for a random scorer equals the positive rate $P/(P + N)$, which makes it interpretable under imbalance—unlike ROC-AUC, whose false-positive-rate axis is normalised by the huge N and therefore compresses precisely the regime that matters.

Operationally, a security team commits to a detection rate and lives with the resulting alert volume. The headline metric of this work is therefore **false alarms per day at fixed recall**: fix recall at 0.90, find the smallest score threshold achieving it, count false positives above the threshold, and divide by the *actual elapsed duration* of the test period.

4.2 Temporal validation and leakage

A model intended to score *future* data must be evaluated on data that is strictly in the future of everything it was trained on. Random shuffling violates this: training and test units become interleaved in time, and the model implicitly exploits information from the test period (shared regimes, shared incidents), inflating measured performance—a form of **data leakage**. The protocol used throughout this work is a **temporal split**: units are ordered by start time; the earliest fraction forms the training set and the strictly later remainder the test set. The prohibition extends beyond labels: normalisation constants, feature vocabularies, and alert thresholds are all computed on training data only.

4.3 Unsupervised detectors

4.3.1 Isolation Forest

Isolation Forest [6] exploits the fact that anomalies are “few and different.” An isolation tree recursively partitions the data with axis-aligned random splits; the number of splits required to isolate a point into its own cell is small for points in sparse regions and large for points inside dense clusters. Averaging path lengths over an ensemble of randomised trees yields an anomaly score. The method is unsupervised, near-linear in sample size, and a standard baseline for tabular anomaly detection—hence its role here as the detector to beat.

4.3.2 Autoencoders

An **autoencoder** is a neural network composed of an encoder $f_\theta : \mathbb{R}^d \rightarrow \mathbb{R}^k$ and a decoder $g_\phi : \mathbb{R}^k \rightarrow \mathbb{R}^d$ with $k \ll d$, trained to minimise the reconstruction error $\|x - g_\phi(f_\theta(x))\|^2$ over a training corpus. The bottleneck dimension k prevents the identity mapping; to reconstruct well, the network must learn the dominant regularities of the training distribution. When trained *exclusively on normal data*, the network becomes an accurate reconstructor of normal patterns; inputs from outside that distribution are reconstructed poorly. The per-unit reconstruction error is thus a natural anomaly score. Compared with the Isolation Forest, the autoencoder can capture non-linear correlations between features (here, co-occurrence patterns of event templates) at the cost of training complexity and hyperparameters.

4.4 Point processes and the Hawkes model

4.4.1 Counting processes and conditional intensity

A temporal **point process** is a random set of event times $0 < t_1 < t_2 < \dots$; equivalently a counting process $N(t) = \#\{i : t_i \leq t\}$. Its law can be specified through the **conditional intensity**

$$\lambda(t) = \lim_{\Delta \downarrow 0} \frac{\Pr[N(t + \Delta) - N(t) = 1 \mid \mathcal{H}_t]}{\Delta},$$

where $\mathcal{H}_t$ is the history of events before t : the instantaneous event rate given the past. The homogeneous Poisson process is the memoryless special case $\lambda(t) \equiv \mu$; its inter-event times are i.i.d. exponential and its counts in disjoint intervals independent. Log streams violate memorylessness flagrantly—events arrive in bursts—which motivates a model in which the past excites the future.

4.4.2 The Hawkes process

The (univariate, exponential-kernel) **Hawkes process** [7] posits

$$\lambda(t) = \mu + \sum_{t_i < t} \alpha e^{-\beta(t-t_i)}, \quad \mu, \alpha, \beta > 0. \quad (4.1)$$

Each event raises the intensity by α ; the excitation decays exponentially at rate β (characteristic memory $1/\beta$). The process admits a branching (Galton–Watson) representation: “immigrant” events arrive at rate μ , and each event, of any generation, independently generates offspring according to the kernel. The expected number of direct offspring per event is

$$\eta = \int_0^\infty \alpha e^{-\beta s} ds = \frac{\alpha}{\beta}, \quad (4.2)$$

the **branching ratio**. By branching-process theory the regime dichotomy is sharp: $\eta < 1$ (subcritical) yields finite cascades and a stationary process with mean rate $\mu/(1 - \eta)$; $\eta \geq 1$

yields explosive, non-stationary behaviour. The branching ratio thus condenses “how cascade-prone is this stream?” into one interpretable number.

4.4.3 Likelihood and estimation

For events $\{t_i\}_{i=1}^n$ observed on $[0, T]$, the log-likelihood of a point process with intensity λ is

$$\ell = \sum_{i=1}^n \log \lambda(t_i) - \int_0^T \lambda(s) ds. \quad (4.3)$$

The first term rewards intensity placed where events actually occurred; the second term is the model’s expected total event count and penalises indiscriminately high intensity. For the exponential kernel the integral is closed-form,

$$\int_0^T \lambda(s) ds = \mu T + \frac{\alpha}{\beta} \sum_{i=1}^n \left(1 - e^{-\beta(T-t_i)}\right),$$

and the sum term admits Ogata’s linear-time recursion: with $A_i = \sum_{j < i} e^{-\beta(t_i-t_j)}$,

$$A_i = e^{-\beta(t_i-t_{i-1})}(1 + A_{i-1}), \quad A_1 = 0, \quad \lambda(t_i) = \mu + \alpha A_i, \quad (4.4)$$

so the full log-likelihood costs $O(n)$ per evaluation rather than the $O(n^2)$ of the naive double sum—the property that makes maximum-likelihood estimation feasible on millions of events. In this work ℓ is maximised over $(\log \mu, \log \alpha, \log \beta)$ (rendering the positivity constraints implicit) with the L-BFGS-B quasi-Newton method.

4.4.4 Goodness of fit: the time-rescaling theorem

Define the **compensator** $\Lambda(t) = \int_0^t \lambda(s) ds$. The **time-rescaling theorem** [9] states that if $\{t_i\}$ is a realisation of the process with conditional intensity λ , then the transformed times $\tau_i = \Lambda(t_i)$ form a homogeneous Poisson process of unit rate; equivalently the rescaled inter-event increments

$$\Delta\tau_i = \Lambda(t_i) - \Lambda(t_{i-1})$$

are i.i.d. $\text{Exp}(1)$. This converts model checking into a one-sample distributional test: compute the $\Delta\tau_i$ under the fitted model and compare them to $\text{Exp}(1)$ via a quantile–quantile plot and a Kolmogorov–Smirnov statistic. Systematic departures localise the model error (e.g. an excess of large increments indicates the model over-predicts activity in quiet periods). Two practical points adopted in this work: the sample mean of $\Delta\tau_i$ must equal 1 under a correctly normalised compensator, providing an independent implementation check; and at $n \sim 10^5$ the KS test has such power that it rejects any parametric model, so the *QQ shape* and the *KS magnitude*, not the *p*-value, are the meaningful diagnostics.

4.4.5 Simulation: Ogata thinning

To validate an estimator one needs data with known parameters. Ogata’s thinning algorithm [8] simulates a Hawkes process exactly: given the current intensity bound $M \geq \lambda(t)$, propose the next event after an $\text{Exp}(M)$ waiting time and accept it with probability $\lambda(t)/M$; accepted events update the intensity. In this project, simulation-based recovery of known (μ, α, β) is a mandatory test that gates any use of the estimator on real data (Section 6.4).

4.5 Extreme-value theory for thresholds

Let X be an anomaly score with distribution function F , and let t be a high threshold. The Pickands–Balkema–de Haan theorem [10] states that for a wide class of F , the conditional law of the exceedance $X - t$ given $X > t$ converges, as t increases, to a **Generalized Pareto Distribution**,

$$\Pr[X - t > x \mid X > t] \approx \left(1 + \frac{\gamma x}{\sigma}\right)^{-1/\gamma},$$

with shape γ and scale σ . The **Peaks-Over-Threshold** method estimates (γ, σ) from observed exceedances and inverts the tail to obtain the level z_q crossed with target probability q :

$$z_q = t + \frac{\sigma}{\gamma} \left[\left(\frac{qn}{N_t} \right)^{-\gamma} - 1 \right], \quad (4.5)$$

where n is the sample size and N_t the number of exceedances above t . Re-estimating (t, γ, σ) on a trailing window while holding q fixed yields a *self-calibrating* alert threshold that tracks distribution drift—the streaming construction popularised by SPOT [11]. The theorem’s hypotheses matter: it concerns *continuous* tails, a point that becomes material in Section 6.6.

Methodology and system design

Two public datasets from the `loghub` collection [2] were used; their complementary properties are summarised in Table 5.1 and were themselves the object of measurement (Section 6.2).

	HDFS	BGL
Origin	Hadoop Distributed File System	Blue Gene/L supercomputer (RAS log)
Raw volume	11,175,629 lines	4,713,493 lines
Time span	$\approx$ 1.6 days	$\approx$ 214.7 days
Labels	per <i>block</i> (2.93% anomalous)	per <i>line</i> (7.39% alerts)
Timestamps	1-second resolution	microsecond resolution
Role	detection & triage	Hawkes timing model

5.2 Pipeline architecture

raw logs $\rightarrow$ **parsing** (Drain) $\rightarrow$ **features** $\rightarrow$ **detection** (AE vs. IsoForest) $\rightarrow$ **thresholding** $\rightarrow$
triage $\rightarrow$ dashboard
 $\searrow$ **Hawkes timing model** (BGL) $\nearrow$

5.3 Log parsing

19

dataset-specific fields (HDFS: the referenced `block_ids`; BGL: the inline label and the node). Using the standard configuration was a deliberate methodological choice: it makes the template vocabulary—and hence all downstream results—comparable with the published literature.

5.4 Feature construction

HDFS: block sessions. Since labels are per block, the unit of analysis is the *block session*: all lines referencing a given block. Each session is represented by its **event-count vector** $x_u \in \mathbb{N}^{48}$, whose j -th entry counts occurrences of template j in the session. Counts are transformed by $x \mapsto \log(1 + x)$ to tame heavy-tailed template frequencies; for the autoencoder they are further standardised with moments computed *on normal training data only*.

BGL: fixed windows with content and timing blocks. BGL has no natural session key, so the stream is segmented into fixed 60-second windows. Each window receives two feature blocks: a *content* block (counts of the top-50 templates, vocabulary fixed on training data), and a *timing* block derived from the Hawkes machinery—the self-excitation level $A(t) = \sum_{t_i < t} e^{-\beta(t-t_i)}$ evaluated along the window and summarised (mean, maximum) at two decay scales ($\beta^{-1} = 1$ s and 30 s), plus the window event count. The timing block uses event *times only*—never labels—so it is a legitimate feature under the leakage rules. A window is labelled anomalous if it contains at least one alert line (evaluation only).

5.5 Validation protocol

The protocol, fixed before any model was trained, is:

1. **Temporal split.** Units ordered by start time; earliest 70% train, latest 30% test; never shuffled. On HDFS this places the boundary at 2008-11-11 06:00:46 (402,543 / 172,518 blocks).
2. **Unsupervised training.** The autoencoder trains on *normal training* units only (389,428 blocks on HDFS); labels never enter training, scaling, or vocabulary construction.
3. **Metrics.** PR-AUC for ranking quality; precision and false alarms per day at recall 0.90 for operations; all durations computed from actual elapsed test time.
4. **Drift.** Every metric is additionally computed on consecutive chronological slices of the test period, with thresholds fixed once on the full test set (an analyst cannot re-tune hourly).
5. **Model-agnostic scoring.** All detectors output scores oriented “higher = more anomalous” and are evaluated by the same code path.

5.6 Engineering and quality practices

The project was developed under version control (git) with incremental, reviewed commits. Correctness of the load-bearing logic is enforced by an automated test suite (34 tests at the time of writing) covering: the metric implementations (operating points, false-alarms-per-day on synthetic cases with known answers), the Hawkes machinery (parameter recovery from simulated data; time-rescaling accepting a true model and rejecting a false one; monotonicity of the compensator), the EVT threshold (exceedance-rate control on simulated stationary and drifting streams), the feature builders, and the triage logic (deduplication, correlation, ranking). A fixed random seed policy makes runs reproducible. The principle applied throughout—*prove components on synthetic data with known ground truth before applying them to real data*—is standard practice in quantitative fields and was treated as non-negotiable here.

Chapter 6

Implementation and results

This chapter follows the pipeline from raw data to dashboard. For each stage it states *what* was built, *why* it was built that way, *how* it works, and what was measured—including the measurements that forced design changes.

6.1 Data acquisition and parsing

Both datasets were downloaded from the public `loghub` mirrors and parsed with `Drain` under the benchmark configurations. The full HDFS log (11.2M lines) parses into **48** templates; BGL (4.7M lines) into **429**. Parsing correctness was verified with a structural invariant: the HDFS template corresponding to block allocation (`NameSystem.allocateBlock`) must occur exactly once per block, and its occurrence count (575,061) matches the number of labelled blocks exactly. An error at this stage would silently corrupt everything downstream, which is why an exact invariant was preferred over spot-checking.

6.2 Exploratory analysis: measurements that shaped the design

Three empirical findings, established before any modelling, redirected the project; we present them as results in their own right.

Finding 1: strong overdispersion (clustering). For a homogeneous Poisson process, the *Fano factor* (variance-to-mean ratio of counts in fixed intervals) equals 1. Measured on per-minute counts, the Fano factor is 5.3 on the HDFS sample and 698–2286 on BGL windows—one to three orders of magnitude above the Poisson value. Event clustering is thus a dominant, quantifiable property of both streams, justifying a self-exciting model *before* any fit (Figure 6.1, left).

Finding 2: HDFS timestamps cannot support a continuous-time model. HDFS records time at 1-second resolution while the stream averages ≈ 80 events/second. Measured consequences: **98.8%** of consecutive events in the global stream carry identical timestamps; within a single block, **90.5%** of consecutive events still tie; every block’s lifetime is ≤ 54 s (median 7s) and occupies a median of ≈ 2 distinct second values. A continuous-time point process requires a strict event ordering that simply is not present. Injecting artificial sub-second jitter was considered and *rejected*: the estimated branching ratio would then measure the jitter, not the system. Consequence: the Hawkes analysis is carried out on BGL, whose microsecond timestamps exhibit 0% ties.

Finding 3: the anomaly rate itself drifts. Under the temporal split, the training period is 3.26% anomalous and the test period 2.16%: the base rate is non-stationary, foreshadowing the drift phenomena measured in Sections 6.3 and 6.6. Figure 6.1 (right) shows the block-session size distribution (median 19 events); the upper tail is disproportionately anomalous, consistent with failing blocks emitting additional retry and error lines.

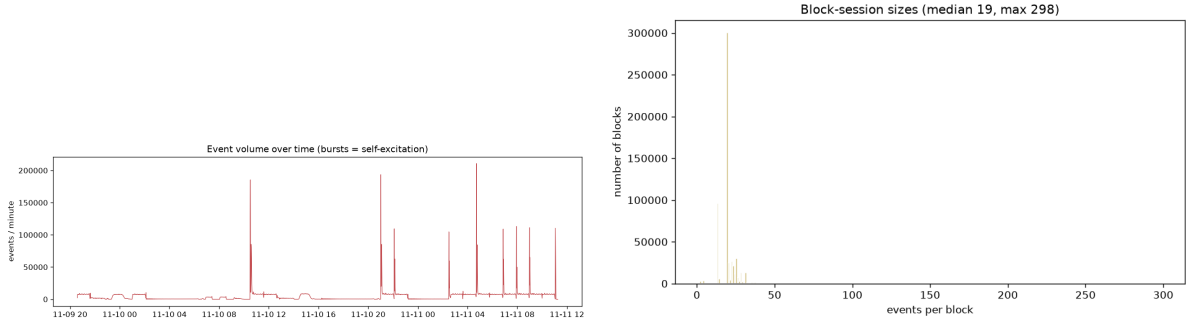


Figure 6.1: Left: HDFS event volume per minute—a flat baseline punctuated by bursts, the visual signature of self-excitation. Right: block-session sizes (median 19, maximum 298).

6.3 Detection on HDFS

Setup. Unit: block session; features: 48-dimensional count vectors (Section 5.4); split and metrics as in Section 5.5. The baseline Isolation Forest is fit on $\log(1+x)$ counts. The autoencoder is a multilayer perceptron $48 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 16 \rightarrow 32 \rightarrow 48$ with ReLU activations, trained with Adam on normal training blocks only, with early stopping on a held-out slice of the normal training data (convergence in ≈ 19 epochs); the anomaly score is the per-block reconstruction mean-squared error.

Results. Table 6.1 and Figure 6.2 summarise the test-set outcome.

Table 6.1: HDFS detection, temporal test split (172,518 blocks, 2.16% anomalous).

Metric	Isolation Forest	Autoencoder
PR-AUC	0.748	0.987
Precision at recall 0.90	0.556	0.961
False alarms / day at recall 0.90	$\sim 12,700$	~ 650

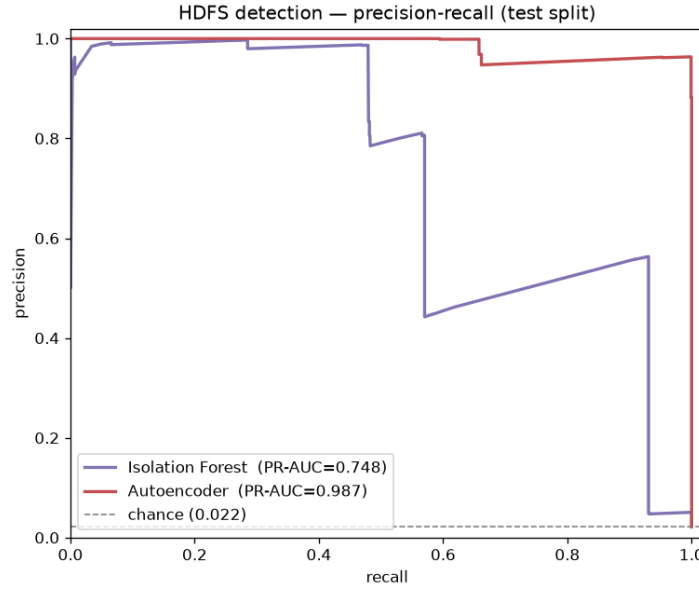


Figure 6.2: Precision–recall curves on the HDFS test split. The autoencoder dominates the baseline at every operating point; the dashed line marks the chance level (0.022).

At the committed 90% detection rate the autoencoder reduces the false-alarm burden by a factor of ≈ 20 . The drift decomposition adds an important nuance: across four consecutive slices of the test window, the baseline’s PR-AUC oscillates between 0.69 and 0.98, while the autoencoder holds ≥ 0.99 throughout—its ranking is not merely better but *stabler*. However, the *false-alarm rate at a fixed threshold* is not stable even for the autoencoder (it varies by orders of magnitude across slices), which motivates Section 6.6.

Positioning

This result reproduces a well-established benchmark outcome (Section 3.3) and is claimed only as validation of the pipeline. Its role in the project is to supply a trustworthy detector and realistic anomaly scores to the components that follow.

6.4 The Hawkes timing model on BGL

Estimator verification. Before any application to real data, the estimator was gated by simulation tests: trajectories generated by Ogata thinning with known (μ, α, β) are refit, and the estimates must recover the truth (achieved within a few percent); the time-rescaling diagnostic must accept the true model and reject a deliberately mis-specified one (both verified). Only then was the estimator applied to BGL.

Fits. Two contrasting 24-hour windows were analysed: a calm day (2005-09-03; 6,858 events) and a severe fault cascade (2005-06-11; 152,669 events). Table 6.2 reports the fits; Figure 6.3 shows the fitted intensity and the time-rescaling QQ-plots.

Table 6.2: Maximum-likelihood Hawkes fits on BGL windows.

Window	Events	$\eta = \alpha/\beta$	KS statistic	mean rescaled gap
2005-09-03 (calm)	6,858	0.971	0.095	1.0001
2005-06-11 (cascade)	152,669	0.9998	0.431	1.0000

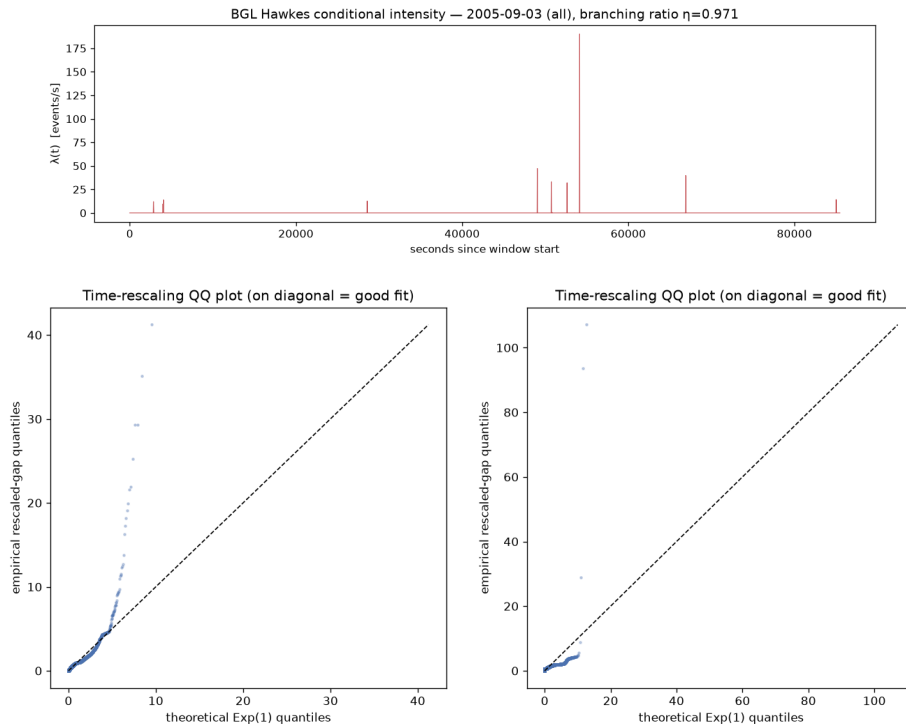


Figure 6.3: Top: fitted conditional intensity $\lambda(t)$ on the calm window —near-zero baseline with self-exciting spikes. Bottom: time-rescaling QQ-plots against $\text{Exp}(1)$ for the calm (left) and cascade (right) windows: the body lies on the diagonal, the upper tail departs.

Interpretation. The estimated branching ratio is *near-critical* on both windows ($\eta \approx 0.97$ – 1.0): each event generates on the order of one direct offspring, i.e. the stream operates at the edge of self-sustaining cascades. This is the project’s central quantitative finding about the data: a precise, validated statement that sharpens the informal observation “logs are bursty.” The time-rescaling diagnostics simultaneously certify the fit’s quality in the bulk (the QQ body lies on the diagonal; mean rescaled gap = 1.0000 confirms correct normalisation) and localise its failure: the upper tail departs upward, meaning the deepest quiet gaps are longer than a single exponential kernel allows. The effect is mild on the calm window (KS 0.095) and severe on the cascade (KS 0.431). Mechanistically, after a burst the fitted excitation predicts continued activity; when silence follows, the integrated intensity over the gap is inflated. A single time scale cannot fit both tight bursts and deep lulls—an argument, developed in Chapter 8, for a multi-scale (sum-of-exponentials) kernel. Given the sample sizes, KS p -values are not informative (the test rejects any parametric model at $n \sim 10^5$); the analysis therefore rests on the QQ geometry and the KS magnitude.

6.5 Ablation: do timing features improve detection?

The Hawkes model earns a place in a *detection* pipeline only if its signal helps. This was tested directly: on 60-second BGL windows, the same Isolation Forest under the same temporal split was trained on three feature sets—content only, timing only, and their concatenation (Section 5.4).

Table 6.3: Feature ablation on BGL windows (test split, 4.71% anomalous).

Feature set	PR-AUC	Precision at recall 0.90	False alarms / day
Content	0.081	0.046	114
Timing	0.087	0.083	61
Content + timing	0.090	0.071	72
Chance	0.047	—	—

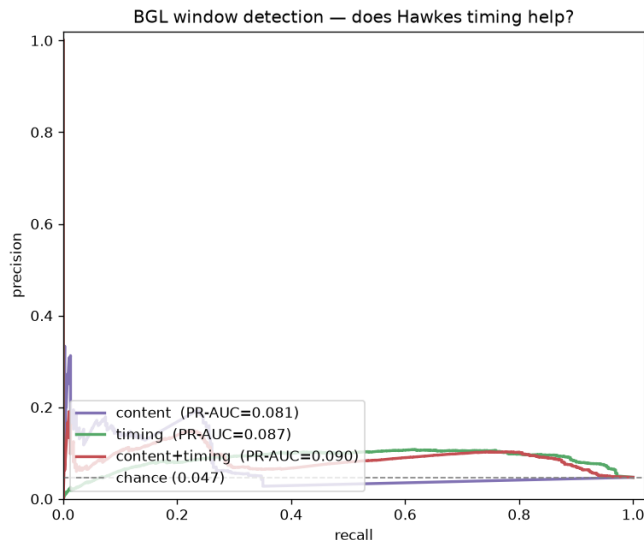


Figure 6.4: Precision–recall curves for the ablation. The content-only curve collapses to the chance line at moderate recall; the timing-based curves retain $\approx 2\times$ chance precision across the operating range.

The result is deliberately reported with both of its faces. In absolute terms, window-level detection on BGL is *hard*: all feature sets remain near PR-AUC 0.09 against a 0.047 chance level, because a 60-second window dilutes a handful of alert lines in heavy normal traffic. Relatively, however, the timing features are the strongest signal available: at recall 0.90 they halve the false-alarm rate with respect to content features (114 $\rightarrow$ 61/day) and nearly double precision. The self-excitation level thus carries operationally useful information that count features do not—the affirmative, if measured, answer to the chapter’s question.

6.6 Adaptive thresholding under drift

The drift decomposition of Section 6.3 identified the deployment problem: with the threshold fixed once, the autoencoder’s false-alarm rate on HDFS swings from 3,664 to 861,686 per day across a five-hour test window—at its worst flagging essentially every block.

EVT implementation and a documented failure mode. The POT threshold of Equation (4.5) was implemented with a method-of-moments GPD estimator (constant-time updates, suitable for streaming) and verified on simulated data: on stationary streams it holds the target exceedance rate; on drifting streams it tracks the drift where a fixed threshold fails. On the real HDFS scores, however, the GPD extrapolation misbehaved—fitted thresholds could fall below the bulk of the data. The diagnosis is a genuine property of the data: **58.7%** of normal test

blocks share a single *identical* reconstruction-error value (normal HDFS sessions are highly repetitive, so entire cohorts have byte-identical count vectors). The resulting score distribution has large atoms, violating the continuity hypothesis of the Pickands–Balkema–de Haan theorem (Section 4.5). Rather than force the tool, the design substitutes a *distribution-free* trailing-window quantile on HDFS and reserves EVT for continuous scores such as the BGL timing features.

Result. With a target flag rate $q = 3\%$, calibrated on the first 20,000 test blocks and run over the remaining 152,518:

Table 6.4: False-alarm stability under drift (HDFS autoencoder scores, $q = 3\%$).

	Fixed threshold	Adaptive (trailing quantile)
False alarms / day (range)	3,664 – 861,686	0 – 7,584
Flagged fraction	up to 100%	$\approx 2\%$, stable
Recall over the period	1.00	0.72

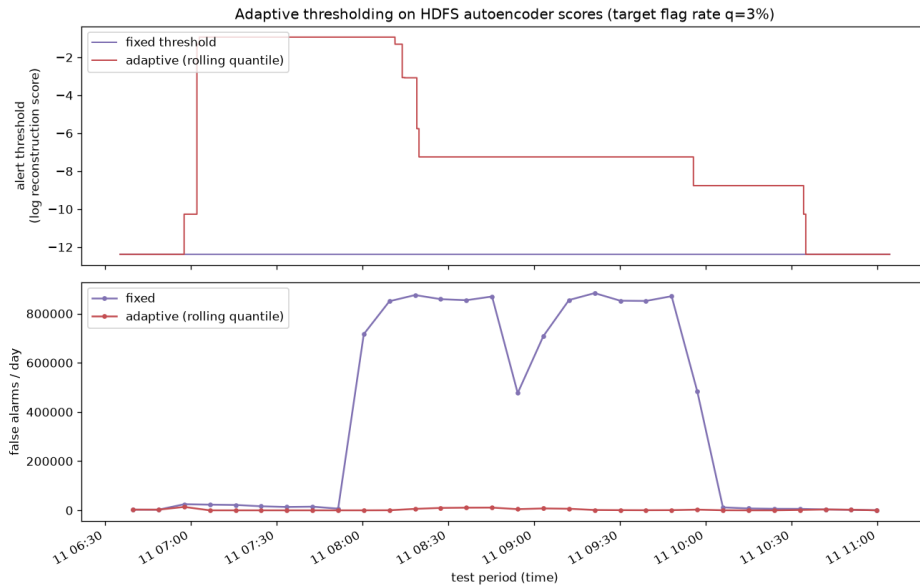


Figure 6.5: Top: fixed versus adaptive threshold against the drifting score distribution. Bottom: resulting false alarms per day; the fixed threshold explodes mid-period while the adaptive threshold holds the alert budget.

The adaptive threshold holds the alert volume within budget throughout the drift, at a transparent cost: when true anomalies exceed the flag budget, recall falls ($1.00 \rightarrow 0.72$ over the period). The trade-off is not hidden but exposed as a single tunable parameter q —exactly the interface an operations team needs.

6.7 Triage: from alerts to ranked incidents

The final analytical stage converts flagged units into analyst-ready incidents in three deterministic-then-generative steps:

1. **Deduplication.** Flags with identical signature (entity and multiset of templates) are merged, retaining a count—repetition is recorded, not re-alerted.

2. **Correlation.** Flags close in time are grouped into a single incident. The grouping rule is the operational translation of the Hawkes analysis: since the stream is near-critically self-exciting, temporally clustered anomalies are overwhelmingly cascade members sharing a root cause, and belong in one incident.
3. **Explanation and ranking.** Each incident is rendered as a strictly-typed JSON object (`priority`, `category`, `summary`, `recommended_action`, `explanation`) and the incident list is ranked. The explanation backend is pluggable: a deterministic rule-based engine (keyword-derived category; priority from severity and cascade size) guarantees a self-contained system, while an LLM backend (Anthropic API with schema-constrained structured output) substitutes richer summaries when credentials are available, falling back automatically otherwise. The LLM is invoked once per *incident*, not per raw flag; the deterministic reduction upstream is what keeps this economical.

On the 200 highest-scoring HDFS test blocks, the agent produced **90 incidents** (P1: 13, P2: 53, P3: 2, P4: 22) across data-loss, replication-failure, and network categories—a $> 2\times$ volume reduction with priorities and actions attached (Figure 6.6).

Honest note

In the development environment no LLM credential was available, so the end-to-end demonstration ran on the rule-based backend; the LLM path is implemented and tested structurally but was not exercised against a live model in this report's runs.

6.8 Demonstrator dashboard

A Streamlit application exposes the pipeline to a non-technical audience in four views: Detection (metrics and PR curves), Thresholding (the drift demonstration), Hawkes (fitted intensity and QQ diagnostics), and Incidents (the ranked incident table with per-incident drill-down). The application was verified headlessly (HTTP health checks and rendered-page screenshots).

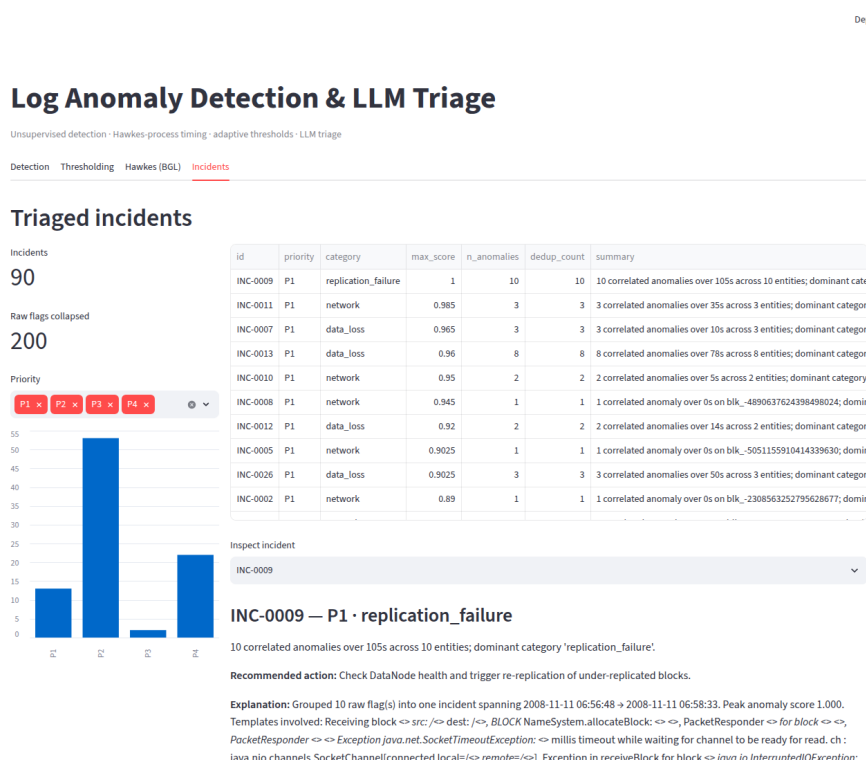


Figure 6.6: The Incidents view of the demonstrator: 200 raw flags reduced to 90 ranked incidents with priorities, categories, and recommended actions.

Chapter 7

Discussion

7.1 Synthesis of results

Table 7.1 consolidates the quantitative outcomes.

Table 7.1: Consolidated results of the internship project.

Item	Result
HDFS parsing	11.2M lines $\rightarrow$ 48 templates (invariant-checked)
BGL parsing	4.71M lines $\rightarrow$ 429 templates
Detection (HDFS test)	PR-AUC 0.987 vs. 0.748 baseline
Operational gain	$\sim 20\times$ fewer false alarms at recall 0.90
Hawkes branching ratio (BGL)	$\eta \approx 0.97\text{--}1.0$ (near-critical)
Timing-feature effect (BGL)	false alarms halved vs. content features
Thresholding under drift	FA/day swing $\times 235 \rightarrow$ held near budget
Triage	200 flags $\rightarrow$ 90 ranked incidents
Software quality	34 automated tests; fully reproducible pipeline

7.2 Limitations

The following limitations are stated with the same prominence as the results.

1. **The HDFS detection figure is a reproduction.** Its value is methodological (pipeline validation), not scientific novelty.
2. **Absolute detection on BGL windows is weak** (PR-AUC ≈ 0.09 against 0.047 chance). The timing result is a relative improvement on a genuinely hard task, and is presented as such.
3. **The single-exponential Hawkes kernel is rejected in the tail** by its own diagnostic; the branching-ratio estimate on the cascade window ($\eta \approx 0.9998$) should be read with this misfit in mind.
4. **EVT was not applicable to the HDFS scores** (discrete atoms); the principled tool was replaced by a distribution-free one there, and its intended domain (continuous timing scores) remains to be exercised in production-like conditions.
5. **The LLM backend was not exercised live** in the reported runs.
6. **Hyperparameters were fixed a priori** (window length, vocabulary size, decay scales, architecture) rather than swept; the reported numbers are therefore conservative.
7. **Public data stands in for OCP data.** The transfer of these methods to an internal log source—with its own formats, volumes, and drift regimes—is an engineering exercise that this project prepares but does not perform.

7.3 Skills developed during the internship

Beyond the technical artefacts, the internship consolidated: the estimation and validation of point-process models (likelihood derivation, simulation, goodness of fit); evaluation methodology for rare-event detection; extreme-value methods and their hypotheses; applied deep learning for unsupervised detection; software engineering practice (packaging, automated testing, reproducibility, version control); and the discipline of documenting negative findings—arguably the most transferable habit of all for a career in quantitative research.

Chapter 8

Conclusion and perspectives

8.1 Conclusion

This internship delivered a complete, reproducible log-anomaly pipeline whose components answer the three structural difficulties of industrial log analysis identified in Chapter 2: unsupervised detection for the absence of labels, a validated self-exciting timing model for the cascade structure of incidents, and calibrated alerting plus LLM-assisted triage for the volume asymmetry. The methodological through-line—prove estimators on synthetic data, validate on strictly out-of-time data, report the operationally meaningful metric, and give negative findings first-class status—is, in the author’s view, the internship’s most important outcome: it is what makes each number in this report defensible.

8.2 Perspectives

Four continuations follow naturally from the limitations:

1. **Multi-scale Hawkes kernels.** Replace the single exponential with a sum of exponentials (fast burst + slow relaxation) or a power-law kernel, targeting the upper-tail misfit exhibited by time-rescaling.
2. **Rescaling residuals as a detector.** The time-rescaling transform converts the fitted model into a calibrated “surprise” signal; large rescaled gaps (or bursts of small ones) are themselves anomaly scores, continuous by construction and therefore in EVT’s domain.
3. **Systematic hyperparameter study** of window length, template vocabulary, decay scales, and detector architecture, with the ablation of Section 6.5 as the evaluation harness.
4. **Transfer to an internal log source** within the host organisation, exercising the LLM triage backend live and measuring analyst time saved—the ultimate criterion for the triage layer.

Bibliography

- [1] P. He, J. Zhu, Z. Zheng, M. R. Lyu. Drain: An Online Log Parsing Approach with Fixed Depth Tree. *IEEE ICWS*, 2017.
- [2] S. He, J. Zhu, P. He, M. R. Lyu. Loghub: A Large Collection of System Log Datasets for AI-driven Log Analytics. *IEEE ISSRE*, 2023.
- [3] S. He, J. Zhu, P. He, M. R. Lyu. Experience Report: System Log Analysis for Anomaly Detection. *IEEE ISSRE*, 2016.
- [4] M. Du, F. Li, G. Zheng, V. Srikumar. DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning. *ACM CCS*, 2017.
- [5] W. Meng et al. LogAnomaly: Unsupervised Detection of Sequential and Quantitative Anomalies in Unstructured Logs. *IJCAI*, 2019.
- [6] F. T. Liu, K. M. Ting, Z.-H. Zhou. Isolation Forest. *IEEE ICDM*, 2008.
- [7] A. G. Hawkes. Spectra of Some Self-Exciting and Mutually Exciting Point Processes. *Biometrika*, 58(1), 1971.
- [8] Y. Ogata. On Lewis' Simulation Method for Point Processes. *IEEE Transactions on Information Theory*, 27(1), 1981.
- [9] E. N. Brown, R. Barbieri, V. Ventura, R. E. Kass, L. M. Frank. The Time-Rescaling Theorem and Its Application to Neural Spike Train Data Analysis. *Neural Computation*, 14(2), 2002.
- [10] J. Pickands III. Statistical Inference Using Extreme Order Statistics. *Annals of Statistics*, 3(1), 1975.
- [11] A. Siffer, P.-A. Fouque, A. Termier, C. Largouët. Anomaly Detection in Streams with Extreme Value Theory. *ACM KDD*, 2017.

Appendix A

Notation

Symbol	Meaning
$\lambda(t)$	conditional intensity of a point process
μ, α, β	Hawkes background rate, excitation jump, decay rate
$\eta = \alpha/\beta$	branching ratio (mean direct offspring per event)
$\Lambda(t)$	compensator $\int_0^t \lambda(s) ds$
$\Delta\tau_i$	rescaled inter-event increment (Exp(1) under a correct model)
(γ, σ)	GPD shape and scale
z_q	threshold exceeded with target probability q
$x_u, y_u, s(u)$	feature vector, latent label, anomaly score of unit u

Appendix B

Reproducibility

All code, tests, figures, and this report's sources are versioned in the project repository. The pipeline is reproduced end to end by the following sequence (Python ≥ 3.10):

```
pip install -r requirements.txt && pip install --no-deps logparser3 && pip install -e .
python scripts/download.py --dataset HDFS --full
python scripts/parse.py --dataset HDFS --input data/raw/HDFS.log
python scripts/build_features.py --input data/interim/HDFS.log_parsed.csv --labels data/raw/anomaly_label.csv
python scripts/run_baseline.py # Isolation Forest
python scripts/train_autoencoder.py
python scripts/run_evt.py --q 0.03
python scripts/run_triage.py --top 200
python scripts/download.py --dataset BGL --full
python scripts/parse.py --dataset BGL --input data/raw/BGL.log
python scripts/fit_hawkes.py --start 2005-06-11 --hours 24 --stream all
python scripts/bgl_experiment.py --window 60 --top-k 50
streamlit run app/dashboard.py
```

The automated test suite is run with `python -m pytest tests/`.